



Variáveis e Tipos

Aula 02 · Bloco 1 — Fundamentos

let, const, os primitivos — e as pegadinhas de tipo do JS

Nesta aula

1. Declarar com `let` e `const`
2. Os **tipos primitivos** e o `typeof`
3. Strings e **template literals**
4. A **coerção de tipos** — aquele `"2" + "3"` da aula passada

Uma variável é uma caixa com etiqueta

```
let idade = 19;           // let: o valor PODE mudar
const nome = "Maria";     // const: o valor NÃO muda

idade = 20;               // ✅ ok
nome = "João";            // ❌ TypeError
```

A regra do curso

Use `const` **por padrão.**

Só use `let` quando o valor
realmente precisar mudar.

Isso torna o código mais previsível — e o `const` avisa
na hora quando você muda algo sem querer.

! E o `var` ?

Em código antigo e em muito tutorial da internet você vai ver `var`.

Ele funciona. Mas tem regras de escopo confusas que **causam bugs**.

Considere o `var` **aposentado**. Neste curso: `let` e `const`.

Nomes de variáveis

```
const nomeCompleto = "Ana Souza"; // ✅ camelCase – padrão do JS e do Java
const nota_final = 8.5;           // ⚠️ funciona, mas não é o padrão
const 2nota = 7;                  // ❌ não pode começar com número
const const = 1;                  // ❌ palavra reservada
```

Use nomes **descritivos**: `mediaDoAluno` conta uma história. `x` não conta nada.

Os tipos primitivos

```
const texto = "entre aspas";    // string
const numero = 42;              // number (inteiro)
const preco = 19.90;            // number – JS não separa int de float!
const ligado = true;           // boolean
const nada = null;              // null: ausência INTENCIONAL
let aindaNaoSei;                // undefined: declarada, sem valor
```

💡 Repare: **um só tipo numérico**. `42` e `19.90` são ambos `number`.

Descobrimos o tipo: `typeof`

```
console.log(typeof "oi");    // string
console.log(typeof 3.14);    // number
console.log(typeof true);    // boolean
console.log(typeof abc);     // undefined
```

💡 **Diferente do Java:** lá você declara `int idade`, `String nome` — o tipo é fixo. Em JS a variável assume o tipo do valor, e pode até trocar de tipo. É a **tipagem dinâmica**: liberdade que exige responsabilidade.

Strings: junte com template literals

```
const nome = "Maria";
const sobrenome = "Silva";

// jeito antigo
console.log("Olá, " + nome + " " + sobrenome + "!");

// jeito moderno – USE ESTE
console.log(`Olá, ${nome} ${sobrenome}!`);
console.log(`Daqui a 5 anos você terá ${19 + 5} anos.`);
```

Repare na **crase** — não é aspas. E o `${}` aceita expressão inteira dentro.

Métodos úteis de string

```
const frase = "JavaScript é divertido";

console.log(frase.length);           // 22
console.log(frase.toUpperCase());    // JAVASCRIPT É DIVERTIDO
console.log(frase.includes("Java")); // true
console.log(frase[0]);               // J
```

⚠ `frase[0]` é o **primeiro** caractere. Em programação, contagem começa em **zero** — você vai reencontrar isso a vida toda.

A pegadinha da coerção

```
2 + 3      → 5
"2" + 3    → "23"
"2" * 3    → 6
"abc" * 3  → NaN
```

O JS **converte tipos sozinho** quando você mistura.

Com **+**, ele escolhe **grudar** o texto.

Com *****, ele escolhe **converter** para número.

Por que isso importa tanto

Tudo que o usuário digita chega como string.

Para converter de propósito:

```
const digitado = "25";           // veio como string
const idade = Number(digitado);  // agora é number
console.log(idade + 5);           // 30 ✓

console.log(Number("abc"));       // NaN
console.log(parseInt("42.9"));    // 42 – só a parte inteira
console.log(parseFloat("42.9")); // 42.9
```

Exercícios da aula

Na pasta `aula-02/` :

1. `ex01.js` — apresentação sua com template literals;
2. `ex02.js` — sua idade em 2026 e em 2050, a partir do ano de nascimento;
3. `ex03.js` — **preveja num comentário** e depois confira: `"5" + 5` , `"5" - 5` ,
`5 + 5 + "5"` , `"5" + 5 + 5` ;
4. `ex04.js` — total de caracteres, maiúsculas e `.includes("a")` no seu nome;
5. **Desafio** 🌶️ `ex05.js` — `"1250.50"` com 10% de desconto, formatado com `.toFixed(2)` .

Próxima aula

Aula 03 — Operadores e Condicionais

Comparar do jeito certo, decidir com `if`,
e a diferença entre `==` e `===`.